

Python Programming

Class Assignment 3

Pandas and Python Classes

Instructions

- For Pandas problems, use the [Online Food Ordering Dataset](#).
- Use Python, NumPy, Pandas, and class concepts covered in class.

Dataset: [Online Food Ordering Dataset](#)

Problems

Problem 1. Explore the Dataset

Load the Online Food Ordering Dataset into a Pandas DataFrame called `df`.

Display:

- First 5 rows
- Number of rows and columns
- Column names
- Data types of all columns

Problem 2. Customer Age Analysis

Using the DataFrame, determine:

- Average age of the customers
- Minimum age
- Maximum age
- Number of male customers
- Number of female customers

Problem 3. Filtering Customers

Find all customers who satisfy both conditions:

- Age is greater than 25
- Occupation is `Employee`

Display the following columns:

- Age
- Gender
- Occupation
- Monthly Income

Problem 4. Occupation Analysis

Use `groupby()` to calculate the average age of customers for each occupation.

Display the result and determine which occupation has the highest average customer age.

Problem 5. Feedback Analysis

Determine the number of customers who gave each type of feedback.

Then calculate the percentage of customers who gave positive feedback.

Display the results clearly.

Problem 6. Occupation and Feedback

Use Pandas to investigate the relationship between occupation and customer feedback.

Group the data by:

Occupation
Feedback

Calculate the number of customers in each group and display the result.

Problem 7. Create a Bank Account Class

Create a class called `BankAccount`.

The class should have the following attributes:

- `account_num`
- `owner`
- `balance`

Use `__init__()` to initialize these attributes.

Create two different bank account objects and display their attributes.

Problem 8. Bank Account Methods

Add the following methods to the `BankAccount` class:

- `deposit()`
- `withdraw()`

The `deposit(amount)` method should add the given amount to the account balance.

The `withdraw(amount)` method should subtract the given amount from the balance only when sufficient balance is available.

Create an account and test both methods.

Problem 9. Withdrawal Status

Modify the `withdraw()` method so that it returns:

- `True` when the withdrawal is successful.
- `False` when the withdrawal cannot be completed.

Test the method with:

- A valid withdrawal
- A withdrawal greater than the available balance

Display the returned values and the account balance.

Problem 10. Display Account Details

Add a `__str__()` method to the `BankAccount` class.

When an account object is passed to `print()`, display its:

- Account number
- Owner
- Balance

Create an account and display it using:

```
print(account)
```

Problem 11. Student Class

Create a class called `Student`.

The class should have the following attributes:

- `name`
- `roll_no`
- `marks`

Create at least three `Student` objects.

Add a method called `average_marks()` that returns the average of the student's marks.

Display the average marks of each student.

Problem 12. Vehicle Class

Create a parent class called `Vehicle`.

The class should have:

- An attribute called `brand`
- A method called `move()`

Create a class called `Car` that inherits from `Vehicle`.

Create a `Car` object and use the inherited functionality.

Problem 13. Method Overriding

Create the following classes:

```
Vehicle
Car
Boat
```

The `Vehicle` class should contain a method called `move()`.

Override the `move()` method in both `Car` and `Boat`.

The methods should produce different outputs for the two classes.

Create one object of each class and call their `move()` methods.

Problem 14. Using `super()`

Modify the `Vehicle` class so that its constructor accepts a `brand` and stores it as an attribute.

Modify the `Car` class so that its constructor accepts:

- `brand`
- `doors`

Use `super()` to initialize the `brand` attribute from the parent class.

Create at least two `Car` objects and display their brand and number of doors.

Problem 15. Polymorphism

Create the following classes:

```
Car
Boat
Airplane
```

Each class should have a method called `move()`.

The `move()` method should behave differently for each class.

Create a list containing objects of all three classes.

Use a single `for` loop to call:

```
vehicle.move()
```

for every object in the list.